

kube-build-app přechod na Go a vývoj metamodelu

Technické shrnutí pro zákazníka a L2 support: co se mění, co zůstává kompatibilní a jak nově zapisovat běžné scénáře v `<app>.yaml`.

Účel změny

Produktizace

Go verze je distribuovatelná jako samostatná binárka pro Linux, macOS a Windows.

Provozní jistota

Jednodušší použití v GitLab pipeline, méně závislostí na runtime prostředí.

Lepší metamodel

Nové bloky pro probes, scheduling a mounts s důrazem na čitelnost.

Cílem není změnit způsob práce se zákaznickými prostředími. Cílem je zjednodušit nasazování, validaci a údržbu.

Co zůstává stejné

- Environment repozitář zůstává hlavním zdrojem konfigurace.
- Adresářová struktura `<env>/apps/*.yaml` zůstává zachována.
- `env.unsecured.json` a `env.secured.json` zůstávají podporované.
- Assety a shared assety zůstávají podporované.
- Stávající YAML soubory jsou zpětně kompatibilní.

L2 support nemusí přepisovat existující konfigurace. Nový zápis je doporučený pro nové změny a postupné čištění.

Co přináší Go verze

Předtím

- Ruby runtime a gem závislosti
- složitější distribuce mimo vývojářské prostředí
- méně vhodné pro Windows stanice
- vyšší riziko rozdílů mezi lokálem a pipeline

Nyní

- jedna binárka `kube-build-app`
- buildy pro Linux, macOS a Windows
- Cobra CLI a kontextová nápověda
- lepší základ pro CI/CD a podporu

CLI pro L2 support a CI/CD

```
kube-build-app validate -e test -R environments  
kube-build-app summary -e test -R environments  
kube-build-app inventory -e test -R environments  
kube-build-app build -e test -R environments -t deploy/test
```

1

validate

rychlá kontrola modelu

2

summary

přehled CPU, paměti, replik
a kontejnerů

3

inventory

strojově čitelný přehled pro
UI a automatizaci

Metamodel není Helm

Záměrně nepřepisujeme Kubernetes do dalšího univerzálního templating systému.

80 %

Běžné deployment scénáře mají mít jasná pole v metamodelu.

20 %

Speciální Kubernetes případy mají jít přes explicitní výjimečný `raw` zápis.

Pravidlo: pokud existuje dedikované pole v metamodelu, použijeme ho. `raw` je výjimka, ne výchozí styl práce.

Proměnné: závazný význam

Zápis	Význam	Kdy použít
<code>{{VAR}}</code>	runtime placeholder	ponechat pro apply-env / deploy fázi
<code>{{env:VAR}}</code>	build-time environment variable	hodnoty z env JSON, .env nebo process ENV
<code>{{var:VAR}}</code>	lokální app variable	hodnota z <code>vars</code> v <code><app>.yaml</code>

Důležité: existující šablony zůstávají kompatibilní. Cílem je pouze jasně pojmenovat odpovědnost jednotlivých scope.

Probes: jak to bylo

Původní zápis: health

```
health:
  http:
    path:
      live: /health/live
      ready: /health/ready
    port: 8080
  delay: 10
  period: 5
  timeout: 3
  success: 1
  failure: 2
```

Původní zápis: probe

```
probe:
  live:
    http:
      path: /actuator/health
      port: 8080
    period: 10
    failure: 5
  ready:
    http:
      path: /actuator/health
      port: 8080
    period: 2
    failure: 2
  start:
    http:
      path: /actuator/health
      port: 8080
    failure: 30
```


Probes: správný nový zápis

Dříve

```
probe:
  live:
    http: { path: /actuator/health, port: "{{var:EXPOSE_PORT}}" }
  ready:
    http: { path: /actuator/health, port: "{{var:EXPOSE_PORT}}" }
  start:
    http: { path: /actuator/health, port: "{{var:EXPOSE_PORT}}" }
    failure: 30
```

Nyní doporučeno

```
probes:
  preset: spring-actuator
  port: "{{var:EXPOSE_PORT}}"
```

Výsledek generuje `livenessProbe`, `readinessProbe` i `startupProbe`. Zápis je kratší a méně chybový.

Probes: přesnější nastavení

```
probes:  
  http:  
    path: /actuator/health  
    port: "{{var:EXPOSE_PORT}}"  
  live:  
    period: 10  
    timeout: 2  
    failure: 5  
  ready:  
    period: 2  
    timeout: 2  
    success: 2  
    failure: 2  
  start:  
    period: 10  
    timeout: 2  
    failure: 30
```

Použijte tam, kde preset nestačí, ale stále nechceme opisovat celý Kubernetes probe objekt.

Scheduling: jak to bylo

Dříve

```
arch: amd64
node_selector:
  workload: tsm
tolerations:
  - key: dedicated
    operator: Equal
    value: tsm
    effect: NoSchedule
```

Omezení

- žádné přirozené místo pro affinity
- žádné jednoduché topology spread
- scheduling pravidla jsou rozptýlená
- složitější rozšiřování metamodelu

Scheduling: správný nový zápis

```
scheduling:  
  arch: amd64  
  node_selector:  
    workload: tsm  
  tolerations:  
    - key: dedicated  
      operator: Equal  
      value: tsm  
      effect: NoSchedule  
  spread:  
    by: hostname  
    max_skew: 1  
    when_unsatisfiable: ScheduleAnyway  
  anti_affinity:  
    self: preferred  
    topology: kubernetes.io/hostname
```

Starý zápis zůstává platný. Nový blok `scheduling` je preferovaný pro nové změny.

Scheduling: raw alternativa pro affinity

```
scheduling:  
  affinity:  
    nodeAffinity:  
      preferredDuringSchedulingIgnoredDuringExecution:  
        - weight: 10  
          preference:  
            matchExpressions:  
              - key: disk  
                operator: In  
                values: [ssd]
```

Metamodel

Běžný případ: architektura, tolerace, node selector, spread, self anti-affinity.

Raw alternativa

Speciální Kubernetes affinity pravidla bez rozšiřování DSL.

Mounts: jak to bylo

Dříve

```
assets:
  - file: assets/app.conf
    to: /app/app.conf
  - temp: true
    to: /app/cache
  - pvc: true
    name: data-claim
    to: /data
  - nfs-server: nfs.local
    path: /export/data
    to: /nfs
  - host-path: /var/lib/host-data
    to: /host
```

Problém

- jen první položka je skutečný asset
- temp/pvc/nfs/host-path jsou volumes
- klíč "to" má více významů
- host-path vypadá stejně bezpečně jako config asset

Mounts: správný nový zápis

```
mounts:  
  - type: config  
    file: assets/app.conf  
    mount_path: /app/app.conf  
  
  - type: empty_dir  
    name: cache  
    mount_path: /app/cache  
  
  - type: pvc  
    name: data  
    claim_name: data-claim  
    mount_path: /data
```

Nový zápis odděluje assets od volumes. Starý blok `assets` zůstává podporovaný kvůli kompatibilitě.

Mounts: NFS, host path a výjimečný raw zápis

NFS a host path

```
mounts:
  - type: nfs
    name: data-nfs
    server: nfs.local
    path: /export/data
    mount_path: /nfs

  - type: host_path
    name: host-data
    path: /var/lib/host-data
    mount_path: /host
```

Výjimečný raw zápis

```
mounts:
  - type: raw
    volume:
      name: special
      projected:
        sources: []
    mount:
      name: special
      mountPath: /app/special
      readOnly: true
```

`host_path` je bezpečnostně citlivý mechanismus. Používat jen při jasném provozním důvodu.

Rollout checksums

Pokud změna konfigurace nemění jméno ConfigMap, ale má vyvolat rollout, použije se anotace nad pod template.

```
rollout_on:
  checksums:
    config:
      files:
        - env.unsecured.json
        - env.secured.json
    mtls:
      files:
        - assets/infrastructure/mtls-gateway-config.tpl
```

Změna obsahu uvedených souborů změní hodnotu anotace `checksum/...`, tím se změní pod template a Kubernetes provede rollout.

Bezpečnost a kompatibilita

Kompatibilita

- staré `health`, `probe`, `assets` fungují dál
- migrace může probíhat postupně
- Go verze drží parity testy nad reálnými vstupy

Bezpečnost

- `host_path` je výjimka
- EncJson zůstává externí bezpečnostní nástroj
- `raw` je řízená výjimka

Doporučený postup pro L2 support

1

Upravit model

Preferovat nové bloky
`probes`, `scheduling`,
`mounts`.

2

Validovat

Spustit `kube-build-app`
`validate`.

3

Zkontrolovat dopad

Použít `summary` a podle
potřeby `inventory`.

Až poté commit a pipeline. Cílem je zachytit chybu dříve než ve fázi deploymentu.

Shrnutí

- Přejechod na Go znamená jednodušší distribuci a provozní použití.
- Nové CLI je vhodnější pro L2 support i CI/CD.
- Metamodel je čitelnější a lépe rozlišuje běžné a speciální případy.
- Zpětná kompatibilita zůstává zachována.
- Doporučený směr pro nové změny: `probes` , `scheduling` , `mounts` .

Výsledek: méně ruční Kubernetes YAML magie, více kontrolovatelného modelu a lepší provozní předvídatelnost.

Děkuji za pozornost

Dotazy k přechodu na Go verzi, metamodelu a doporučenému způsobu práce v environment repozitářích.